

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

6: Lists, Iterators and Operators

Jan H. Mikkelsen
Cyber og Computerteknologi
2. semester – KBH – F26

AGENDA

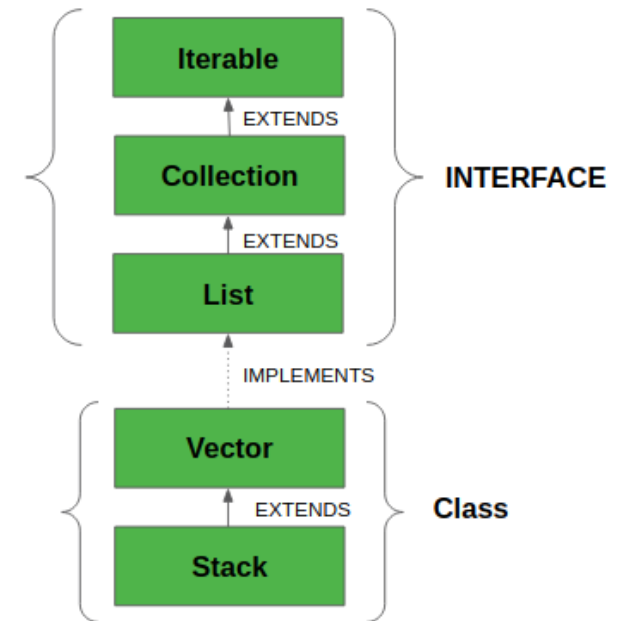
- List interface and using lists
- Iterators
- Operators and overloaded operators
 - Addition, comparisons, equality etc.
- Collection and sorting
- The important stuff
- Assignments



- To be able to understand and use the list interface
- To be able to understand and use iterators
- To be able to understand and use operators and operator overloading

MAKING USE OF INHERITANCE AND INTERFACES

- Designing data structures
- A stack is a kind of Vector
- A Vector is some sort of List
- A List is also a Collection
- A Collection is something we can iterate
- At some time, someone decided this is the model Java takes
- Remember, ALL of these are just models
 - A reflection of someone's mind and view of the world



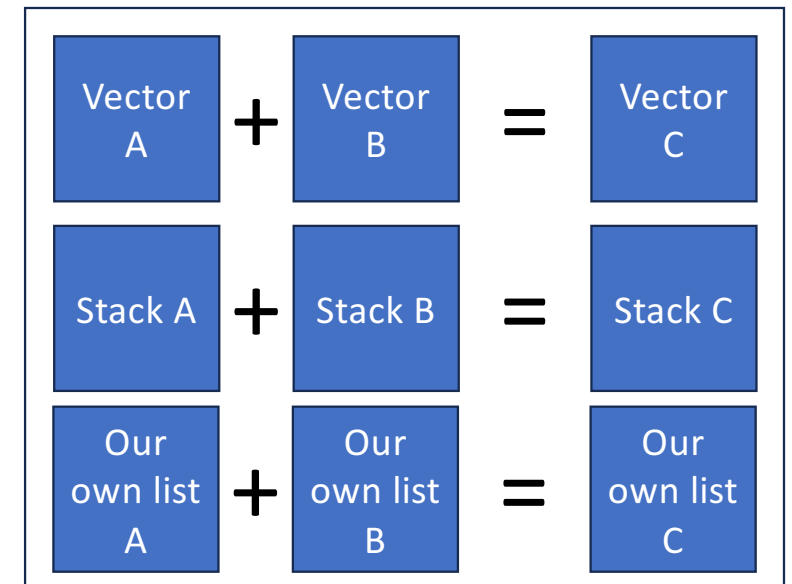
<https://www.geeksforgeeks.org/java-util-vector-class-java/>

LISTS

- The 'List' is an interface and not a class
- Since 'List' is an interface, we know that they all implement the same methods, even though the code behind may be (very) different from class to class
- This ensures that we can use classes of types 'Lists' the same way
- The 'List' interface provides methods such as
 - Add()
 - Contains()
 - Get()
 - Iterator()
 - Remove()
 - Size()
 - Sort()
 - SubList()
 - toArray()
 - and more...

USING THE LIST INTERFACE

- The List interface defines a set of expected methods a class needs to implement
- Makes any use of child classes of list simple to use once we learned one
- We can create our own class, and inherit the List, and others would intuitively know how to use it
 - More or less intuitively – as designers we can still ruin this “intuitively”



USING THE LIST INTERFACE

- A few examples of classes, some of which we have visited previously, that is implementing the List interface
 - ArrayList
 - LinkedList
 - Stack
 - Vector

```
List fruits = new ArrayList();  
fruits.add("Apple");  
fruits.add("Orange");
```

```
Vector<Integer> v = new Vector<Integer>(int size);  
v.add(3);  
v.add(5);
```

```
List<Vehicle> train = new ArrayList<Vehicle>();  
list.add(new Locomotive(...));  
list.add(new Wagon(..));  
Vehicle[] trainAsArray = list.toArray();
```

USING THE LIST INTERFACE

- Common way to access elements in a Collection (e.g. Vector)

```
For (int i=0; i<minVector.size(); i++){  
    minVector.get(i) = ....  
}
```

- What if we wish to remove one of the items, while in the loop?
 - Say, item nr. 3 in a Vector with 10 elements

```
For (int i=0; i<minVector.size(); i++){  
    if (i==3){ minVector.remove(3);}  
}
```

- However, suddenly minVector.size() is no longer 10
 - This is a problem in some cases, and resulting code can be unpredictable – which sucks...

AGENDA

7

- List interface and using lists
- Iterators
- Operators and overloaded operators
 - Addition, comparisons, equality etc.
- Collection and sorting
- The important stuff
- Assignments

ITERATORS

- Iterators are technically speaking an interfaces that are implemented in all Collection sub-classes
- Iterators allows us a simpler and less error prone interaction with elements in any Collection

```
minIterator = minVector.iterator();  
while (minIterator.hasNext){  
    it.next() = ...  
    if (it.id="someIDNo3"){ it.remove(); }  
}
```

Iterator:
hasNext()
next()
remove()
forEachRemaining()

- Also, iterators allows us to work with the current object without having to access it through some method (e.g. minVector.get(i))
- Code generally becomes more readable – improved code quality

ITERATORS – THE FOR-EACH LOOP

- Implementing this interface makes our code read nicer
 - Not strictly important for the computer, but important for us as coders

```
void stopAll(List<Vehicle> train) {  
    for (Iterator<Vehicle> trainSet = train.iterator(); trainSet.hasNext(); )  
        trainSet.next().stop();  
}
```

Versus the following code

```
void stopAll(List<Vehicle> train) {  
    for (Vehicle wagon : train)  
        wagon.stop();  
}
```

The ':' in the for () reads a 'in' and realises a for-each construct

ITERATORS – COMMON MISTAKES

- Look at this code – what is the error?
 - If you attempt to run it, it will throw an `NoSuchElementException`
- The error is that we advance the outer, `i.hasNext()`, inside the inner loop
- Here the outer iterator is advanced for every inner iteration – so same mistake
- These kinds of errors are really difficult to catch

```
List suits = ...;  
List ranks = ...;  
List sortedDeck = new ArrayList();  
  
for (Iterator i = suits.iterator(); i.hasNext(); )  
    for (Iterator j = ranks.iterator(); j.hasNext(); )  
        sortedDeck.add(new Card(i.next(), j.next()));
```

```
List locomotives = ...;  
List wagons = ...;  
List trains = new ArrayList();  
  
for (Iterator locIter = locomotives.iterator(); locIter.hasNext(); )  
    for (Iterator wagIter = wagons.iterator(); wagIter.hasNext(); )  
        trains.add(new Train(locIter.next(), wagIter.next()));
```

ITERATORS – COMMON MISTAKES

- Now look at these two pieces of code, what seems to be the difference?

```
for (Iterator locIter = locomotives.iterator(); locIter.hasNext(); ) {
    Locomotive locomotive = (Locomotive) locIter.next();
    for (Iterator wagIter = wagons.iterator(); wagIter.hasNext(); )
        trains.add(new Train(locomotive, wagIter.next()));
}
```

Vs.

```
List locomotives = ...;
List wagons = ...;
List trains = new ArrayList();

for (Iterator locIter = locomotives.iterator(); locIter.hasNext(); )
    for (Iterator wagIter = wagons.iterator(); wagIter.hasNext(); )
        trains.add(new Train(locIter.next(), wagIter.next()));
```

- (A) advances outer iterator once per outer iteration, while (B) advances the outer iterator inside the inner loop
- And then look at this code

```
for (Locomotive locomotive : locomotives)
    for (Wagon wagon : wagons)
        trains.add(new Train(locomotive, wagon));
```

implements

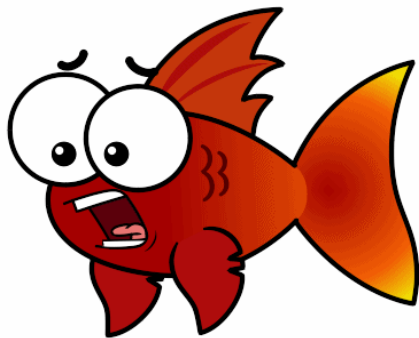
```
for (Iterator<Locomotive> locIter = locomotives.iterator(); locIter.hasNext(); ) {
    Locomotive locomotive = locIter.next(); // advance once per outer iteration
    for (Iterator<Wagon> wagIter = wagons.iterator(); wagIter.hasNext(); ) {
        Wagon wagon = wagIter.next(); // advance once per inner iteration
        trains.add(new Train(locomotive, wagon));
    }
}
```

ITERATORS – IN CONCLUSION

- Using interfaces properly, we can greatly enhance code readability
- Using inheritances, we can greatly enhance code readability
- Using comparisons, we can greatly enhance code readability
- It does not matter for the computer
- At the end of the day, factors impacting performance
 - The compiler's ability to make code efficient – out of scope for many of us
 - Choice of algorithms that we use for particular problems
 - Our coding style and approach
 - we can do things the efficient or the dumb way
 - Reduction of coding errors – directly related to code readability!!

BREAK – PAUSE(FISK)

13



Jan H. Mikkelsen, OOADI6, CoCT, 2026

AGENDA

14

- List interface and using lists
- Iterators
- Operators and overloaded operators
 - Addition, comparison, equality etc.
- Collection and sorting
- The important stuff
- Assignments

OPERATION ON DATA

- For simple data types some operations are also simple
 - =, <, >, increment, decrement etc.
- What about more complex data types?
 - Strings, Vectors, Queues
- And even worse, our own homebrewed classes?

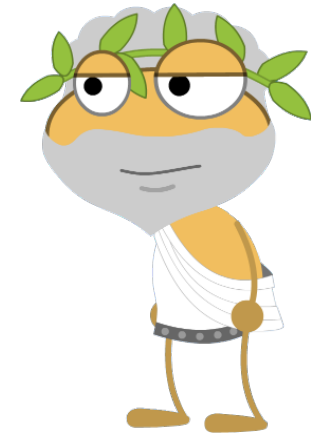
OPERATIONS

- It is clear from our math classes what this does

$$A^2 + B^2 = C^2$$

- We also have an idea what this should do

"I am some" + " text that" + " is concatenated."



- We use "+" in both cases, but certainly *numbers are not strings*
- So how would Java (or any other language for that matter) know the difference?

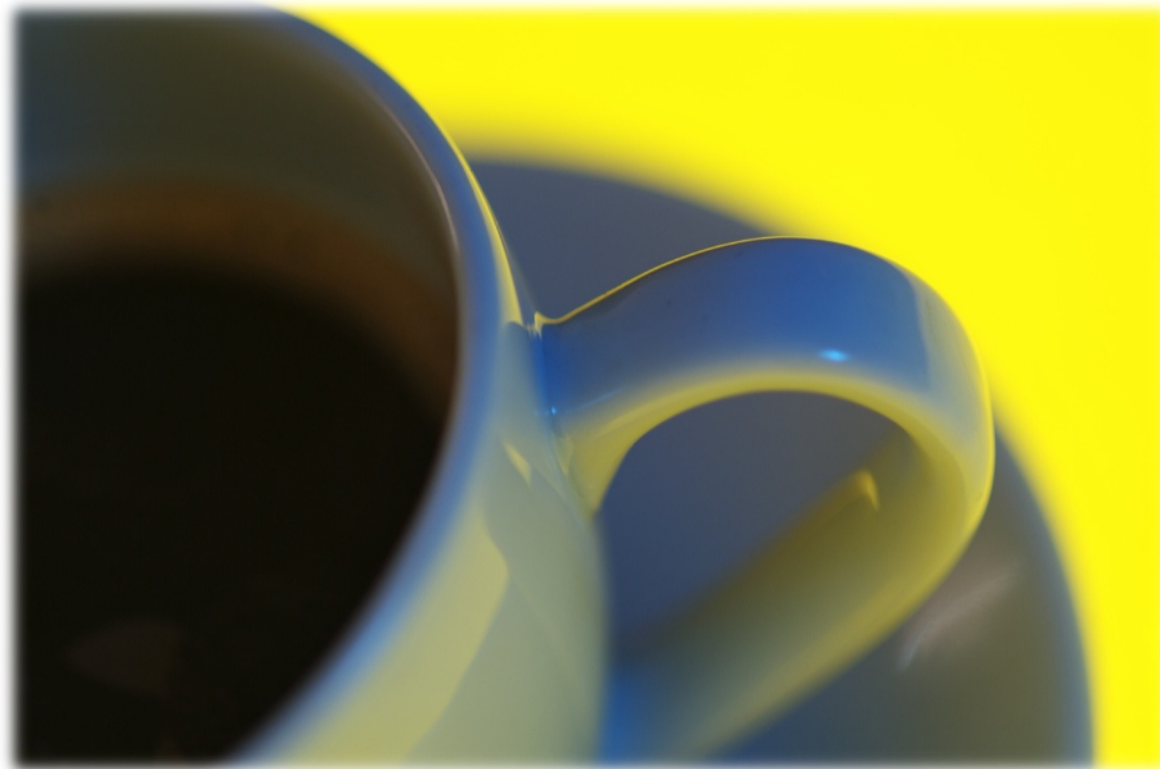
OPERATIONS – WHAT DOES “+” IMPLY?

- In Java, the meaning of the “+” token depends on the **operand types** and the **context**
- Java **does not allow user-defined operator overloading** (unlike C++), but the language itself defines a few overloads of “+”
- Limitations to operator overload in java due to design philosophy
- This reduces complexity and limits risk of surprises—favoring explicit method calls on your own types instead
- This modus operandi is handy but also comes with a few “gotchas” cases

```
System.out.println("sum: " + 1 + 2); // Results in "sum: 12"  
System.out.println("sum: " + (1 + 2)); // Results in "sum: 3"
```

"BIKER BREAK!"

18

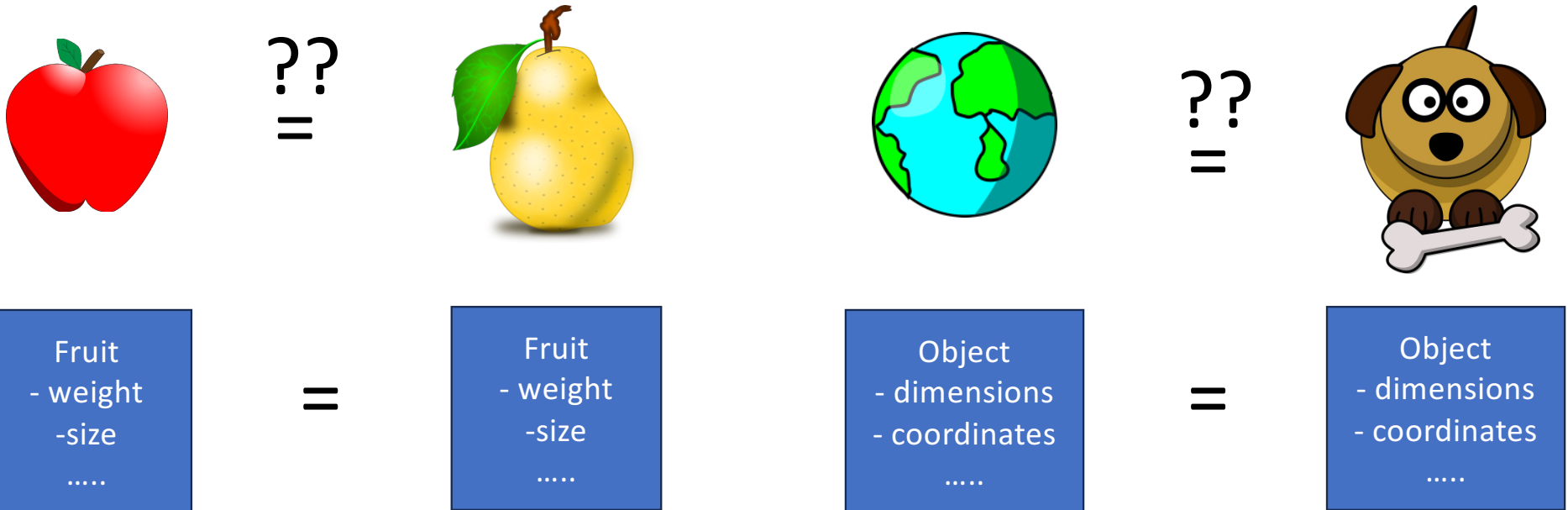


Jan H. Mikkelsen, OOADI6, CoCT, 2026

COMPARISONS

19

- How do we do the following comparisons?
 - Why? Because my boss told me to do so!!!



EQUALITY

- When we want to test for equality we need to first ask, what it is we want to achieve?

What does *equal* actually mean to us?

- If we do nothing, Java (and most other languages) assumes that an equal test is the same as testing if two objects are the same instance
- For instance, if we want to check if `house1 == house2`
 - `House@d4e332ee == House@45eab2c3` ? No, they not equal
 - `House@d4e332ee == House@d4e332ee` ? Yes, they are equal

EQUALITY

- Consider a class `Coordinate` containing longitude and latitude

What does *equal* actually mean to us?

- When is `Coord_1 == Coord_2` ?
 - Is it when `longitude_1 == longitude_2`?
 - No, we need also `latitude_1 == latitude_2`
 - However, we use floating points for long, latt coordinates, so the two equals are extremely rarely true
 - Insted, we need to define "a within area", such that
 - $\text{Sqrt}(\text{diff_long}^2 + \text{diff_latt}^2) < \text{Range}$: assumes Euclidian system
 - Much more complex if we need to consider Earth curvature.... Left as an excercise to you!
- So, we may have different options in considering what is equal
 - We want to hide equal under the hood

EQUALITY

- If, instead, we just want to know if an object is a Location type?
- Here, Java has a word `instanceof`, which can be used to do this test

```
House house = new House();  
If (house instanceof Location){  
    .... We can now safely use Location methods  
}
```

- This gives a safety in knowing that our objects are of a certain class, and that it, within a specific scope, can use specific methods
- Alternatively, we can also “downcast” in the above example
 - `Location locationOfHouse = (Location) house;`
 - However, if `house` is not a `Location`, then we get an error (exception - which we will deal with later)

EQUALITY

- In addition, all java classes inherits from the Object class
- This class has a default equals() method where the default behaviour is comparing object hashes (as we saw)
- Overwriting this method enables us to write specific interpretations of equal
- We also avoid having to write code like this for each compare we do

$$d = 2R \times \sin^{-1}(\sqrt{[\sin^2((\theta_2 - \theta_1)/2) + \cos\theta_1 \times \cos\theta_2 \times \sin^2((\phi_2 - \phi_1)/2)]})$$

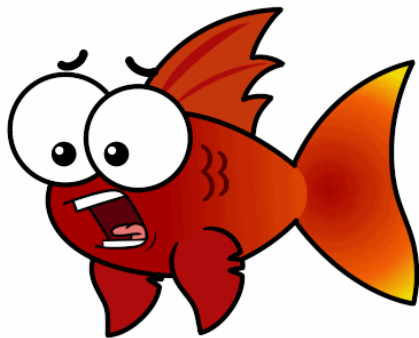
- Also, there are some sign issues that needs to be considered – and it assumes a perfect round Earth.

COMPARATIVES – LARGER AND SMALLER

- Similar problems arises with comparisons
 - What does this mean : `Coordinate_1 > Coordinate_2`?
 - We may need this for sorting coordinates (most sorting algorithms are based on comparisons)
- Use case specific comparisons
 - Once we agree on a comparison method, we can hide it
 - Naming methods to do compare or use the operator
 - In some cases, we may want to implement different ways to compare
 - `isNorthOfLocation()`; `isWestOfCoordinate()`; `distanceIsFurtherThan()`
- Whatever we do – PLEASE name your method so it makes sense
 - Do NOT use e.g. - `cmpN()`; `cmpW()`; `dift()`
 - In less than a month **you** have forgotten what `dift()` does and your co-workers have no clue either. And then **you** need to look into the code and derive

BREAK – PAUSE(FISK)

25



Jan H. Mikkelsen, OOADI6, CoCT, 2026

AGENDA

26

- List interface and using lists
- Iterators
- Operators and overloaded operators
 - Addition, comparison, equality etc.
- Collection and sorting
- The important stuff
- Assignments

THE COLLECTIONS CLASS

- This class contains a lot of useful static methods that operates on Collections (Collection is a super interface to List)
 - If we have a List, then we also have a Collection, and as a consequence, we can use the Collections class for List objects
- Some of the useful methods that are found in Collections include
 - `asLifoQueue(Deque <T>)` – returns a view of a Deque a LIFO queue
 - A Deque is an interface implemented by e.g. the LinkedList datatype
 - `frequency(Collection<?> x, Object o)` – count the number of type object
 - `Max(Collection <? Extends T> coll, Comparator<? Super T> comp)`
 - T is a type of class
 - `Sort(List<T> list, Comparator <? Super T> comp)`

THE COMPARATOR INTERFACE

- An interface that is implemented to ensure sorting of some collection of objects.
- Since sorting of various objects needs to be done differently for different objects, implementation also needs to be specific
- Care must be taken to ensure consistency for the ordering

The ordering by a comparator c on a set S is said to be consistent with equals iff $c.compare(e1, e2) = 0$ if $e1.equals(e2)$ for every $e1$ and $e2$ in S

- If we can guarantee this in our implementation of a comparator interface, then we can also safely use the sorting in the Collections class

SORTING WITH THE COLLECTIONS CLASS

- Using Collection class `sort()` we may sort stuff without coding sorting algorithms (defaults to an adaptive merge-sort)
 - Time complexity is $O(n \log(n))$
 - Space complexity is $O(n)$
- `Arrays.sort()` uses a dual-pivot quick-sort algorithm
 - Quicksort (non-stable)
 - Timsort (stable)
 - Both time complexity $O(n \log(n))$
- Writing sorting algorithms based on the above base classes is doable

```
class Sortbyroll implements Comparator<Student> {
    // Used for sorting in ascending order of
    // roll number
    public int compare(Student a, Student b)
    {
        return a.rollno - b.rollno;
    }
}
```

```
class Main {
    public static void main (String[] args) {
        ArrayList<Student> ar = new ArrayList<Student>();
        ar.add(new Student(111, "bbbb", "london"));
        ar.add(new Student(131, "aaaa", "nyc"));
        ar.add(new Student(121, "cccc", "jaipur"));

        System.out.println("Unsorted");
        for (int i=0; i<ar.size(); i++)
            System.out.println(ar.get(i));

        Collections.sort(ar, new Sortbyroll());

        System.out.println("\nSorted by rollno");
        for (int i=0; i<ar.size(); i++)
            System.out.println(ar.get(i));
    }
}
```

AGENDA

30

- List interface and using lists
- Iterators
- Operators and overloaded operators
 - Addition, comparison, equality etc.
- Collection and sorting
- The important stuff
- Assignments

THE IMPORTANT STUFF

- Lists store elements in ordered fashion - always use generics and avoid raw types
- Iterators provide a safe way to visiting list elements one at a time by enforcing correct traversal
 - Never call next() more times than hasNext() returns true
- When possible, use enhanced-for loops to prevent most iterator bugs
- Operator “+” is overloaded by the language (numbers + strings) and not by users
- Java forbids (unlike C++ and Python) user-defined operator overloading
 - This is a deliberate design choice to achieve simplicity, readability, and avoiding “clever” but confusing operator behaviour – keeps code clear

AGENDA

32

- List interface and using lists
- Iterators
- Operators and overloaded operators
 - Addition, comparison, equality etc.
- Collection and sorting
- The important stuff
- Assignments



ASSIGNMENTS

- 1) Create a list of trains from your previous assignments
- 2) Create methods to allow you to sort the trains, for example
 - a. According to type
 - b. According to length
 - c. According to weight
- 3) Create a small program that allows you to type in a query for a train (and/or specific wagons) and return this. Implement this so the line interface allows use of larger than, smaller than, equal to type of queries. Search should be for example
 - a. Search by type of trains (probably returns a list of those types or null)
 - b. Search by length
 - c. Search by weight
 - d. Has electric power drive (electric trains)